

Agent 项目设计参考

0. 这份文档是干什么的

这份文档不是继续分析 Claude Code Sourcemap 的源码细节。

它做的事情更直接：

把这个项目里对开发 Agent 真正有迁移价值的经验，提炼成一套以后看别的 Agent 项目、设计别的 Agent 系统时都能复用的参考框架。

所以这份文档不是“这个项目做了什么”，而是：

- 什么做法值得学
- 为什么值得学
- 适合什么阶段的 Agent 项目
- 如果不用，会踩什么坑
- 如果上得太早，又会不会变重

如果以后我要给其他 Agent 项目提建议，我会优先拿这份框架判断：

1. 它到底是聊天壳，还是 Agent Runtime。
2. 它缺的是模型能力，还是系统能力。
3. 它现在该补的是 Prompt、Tool、Workflow、Memory，还是组织层。

1. 最重要的总原则：先把 Agent 当“运行系统”，不要先当“提示词工程”

这个项目最值得学的第一件事，就是它没有把 Agent 理解成：

- 一段很强的 system prompt
- 几个 function calling
- 一个会话 UI

它真正做的是一个 Agent Runtime。

这里的 Runtime，你可以把它理解成：

- 请求怎么进来
- 上下文怎么组装
- 模型怎么做决策
- 工具怎么落地执行
- 结果怎么回流到下一轮推理
- 长会话怎么不崩
- 记忆怎么沉淀
- 多 Agent 怎么协作

为什么这件事重要？

因为很多 Agent 项目做到后面失败，不是 Prompt 不够强，也不是模型不够新，而是系统没把“持续工作”这件事接住。

常见表现是：

- 能演示，不能稳定干活
- 能做一轮，做不了长链路任务
- 能调工具，但副作用不受控
- 能开子 Agent，但收不回来

所以以后只要看一个 Agent 项目，我最先会判断：

它有没有明确的运行时主线，还是只是把模型、工具、UI 拼在一起。

2. 真正的系统中心应该是 query loop，不是模型 API

这个项目最清楚的一点，是它把系统中心放在 query loop 上。

也就是：

1. 准备上下文。
2. 让模型判断下一步。
3. 如果模型要行动，就执行工具。
4. 把结果重新注入上下文。
5. 继续判断，直到这一轮结束。

这看起来很基础，但其实是 Agent 和普通聊天系统的根本分界线。

2.1 为什么这比“模型调用”更重要

如果系统中心只是模型 API，那么工具、记忆、compact、多 Agent 都会变成零散外挂。

一旦这样做，后面通常会出现：

- 不同能力接入方式不统一
- 工具结果没有一致的回流协议
- 长会话逻辑只能硬补
- 记忆提取和主对话脱节

而如果中心是 query loop，那么这些问题会自然有落点：

- Prompt 是 loop 的输入控制器
- Tool 是 loop 的行动执行器
- Compact 是 loop 的上下文治理器
- Memory 是 loop 的长期补给系统
- Multi-Agent 是 loop 的执行体扩展机制

2.2 给其他项目的建议

如果一个项目还很早期，不需要一开始就做得很重，但至少要先把这件事设计清楚：

- 一轮请求的完整状态机是什么
- 工具结果如何返回给下一轮推理
- 哪些后处理动作发生在 turn 结束后

只要这一步不清楚，后面所有“高级能力”都会变得很脆。

3. Prompt 不是一段话，而是一套上下文供给系统

这个项目对 Prompt 最值得迁移的认知，不是写法，而是结构。

它提醒我们：

Prompt 工程真正难的，不是把话写得厉害，而是把上下文供给得稳定。

3.1 应该怎样拆 Prompt

至少要区分三类信息：

- 稳定规则：长期不变的行为约束、工具原则、输出规范
- 动态配置：当前模式、权限、环境、可用能力
- 临时上下文：本轮提醒、短期记忆、附件、局部任务要求

这个项目通过 system prompt sections、attachments、动态拼装来处理这些差异。

3.2 为什么这样更好

因为如果全部塞进一个大 Prompt，会有几个问题：

- 动态信息一变，缓存全失效
- 很难知道某种行为是由哪段 Prompt 驱动的
- 小改一个规则，可能把整段上下文都搞乱
- 长会话里 Prompt 维护会越来越痛苦

3.3 对其他 Agent 项目的启发

以后如果我给别的项目提建议，我会先看它的 Prompt 有没有做到这些最低标准：

- 稳定规则和动态信息有没有拆开
- 是否支持按片段注入而不是全量重拼
- 是否考虑了缓存命中
- 是否给 Prompt 设计了“可调试性”

3.4 什么时候不需要做这么重

如果项目还处于 very early demo 阶段，且目标只是验证单一场景，完全可以先用一段大 Prompt。

但一旦出现下面任意一个信号，就该开始结构化：

- 会话开始变长
- 模式开始变多
- 工具开始增多
- 多角色 Agent 开始出现
- 想做 Prompt cache 或成本优化

4. Tool Calling 真正难的不是“能调用”，而是“副作用治理”

这是这个项目里我觉得最值得所有 Agent 团队学的部分之一。

很多团队对工具调用的理解还停留在：

- 模型输出函数名和参数
- 代码执行函数
- 把结果返回模型

这只能算是最初级的 Function Calling，不是成熟的 Tool Runtime。

4.1 一个成熟 Tool Runtime 至少要管什么

- 工具定义与能力边界
- 参数校验
- 权限决策
- 并发安全
- 失败处理
- 副作用可见性
- 结果回流协议

这个项目通过 `Tool.ts`、`toolorchestration.ts`、权限层、MCP 接入层，把这些问题系统化了。

4.2 为什么副作用治理这么关键

因为真正危险的不是“模型答错了”，而是“模型做错了动作”。

尤其是以下类型的工具：

- 文件写入
- 命令执行
- 外部 API 调用
- 多工具并发操作
- 触发后台任务

如果系统没有统一治理层，就容易出现：

- 工具调用顺序错乱
- 并发写导致状态污染
- 权限绕过
- 同一类结果无法统一回写

4.3 给其他项目的建议

以后看到一个 Agent 项目，我会问几个非常具体的问题：

1. 工具返回值是不是最终会被包装成统一消息协议？
2. 系统有没有区分“纯读工具”和“有副作用工具”？
3. 并发执行是拍脑袋开的，还是有并发安全判断？
4. 权限判断是在工具外统一做，还是散落在各工具内部？

如果这四个问题都没有明确答案，这个项目的工具层大概率还停留在 demo 级。

4.4 什么时候 MCP 特别有价值

MCP 的价值不只是“接更多工具”，而是把外部能力接入统一语义层。

它更适合这些场景：

- 工具来源很多，不只本地函数
- 希望能力可以插拔
- 不想把每个外部工具都硬编码进主程序
- 需要在不同 Agent 或不同运行模式下复用工具生态

如果项目只是几个本地函数，MCP 可以暂时不上。

但只要工具生态开始扩大，标准化协议会越来越重要。

5. Workflow 编排要按“时间尺度”分层，而不是只画步骤图

这个项目对 Workflow 最大的启发是：

Workflow 不是“先 A 再 B 再 C”，而是不同时间尺度上的任务治理。

它至少分成了几层：

- 单轮 query loop
- 会话级编排
- 背景任务编排
- 多 Agent 编排

5.1 为什么这种分层比普通流程图更强

因为很多 Agent 项目的流程图只讲了单轮主路径，却没讲清楚：

- 长会话如何维护
- stop hooks 何时触发
- compact 之后状态如何清理
- memory extraction 为什么要放到后台
- 多 Agent 的状态如何回流

结果就是图画得很完整，系统跑起来却到处断层。

5.2 给其他项目的建议

以后设计 Workflow，我会优先让团队分别回答四个问题：

1. 单轮请求怎么闭环？
2. 会话过长怎么治理？
3. 哪些事必须异步到后台？
4. 多执行体之间怎么传状态和收结果？

如果这四件事没有分别设计，说明 Workflow 还只是“Happy Path 图”，不是可运行架构。

5.3 什么时候这种分层会显得太重

对于非常小的单 Agent、单轮工具型产品，分这么细会显得复杂。

那时可以先只有两层：

- 主 query loop
- turn-end hooks

但一旦你开始遇到下面这些问题，就该升级了：

- 上下文太长
- 异步任务越来越多
- 需要长期记忆
- 任务拆分给其他 Agent

6. RAG 不一定先做成向量库，先解决“需要什么被取回来”

这个项目对 RAG 的启发很大，因为它说明了一件很容易被忽略的事：

RAG 的本质不是“上向量库”，而是“让真正相关的信息在正确时间回到上下文里”。

它现在的做法更像混合型检索：

- 规则型检索
- 相关记忆检索

- 工具检索
- 外部资源检索

6.1 为什么很多团队会过早向量化

因为“做 RAG”这三个字太容易被理解成：

- 切 chunk
- 生成 embedding
- 建索引
- top-k 检索

但很多 Agent 项目真正需要的，根本不是重型语义检索，而是：

- 把规则稳定注入
- 把历史偏好按条件召回
- 把工具使用说明在需要时拿出来
- 把外部搜索结果接进当前任务

6.2 给其他项目的建议

以后我会先建议团队先问：

1. 要检索的知识到底分几类？
2. 哪些是规则，哪些是经验，哪些是外部事实？
3. 哪些知识必须高精度，哪些允许宽松召回？
4. 没有向量库，能不能先用结构化检索解决 60% 的问题？

如果这些问题都没想清楚，就先上向量库，往往会把系统做重但不一定做对。

6.3 什么时候该升级成更重型 RAG

出现这些迹象时，可以考虑升级：

- 知识体量大幅增长
- 检索对象高度非结构化
- 需要跨文档语义召回
- 规则、经验、资料库之间需要统一排序融合

但升级也应该是在现有混合检索之上演进，而不是把已有结构化能力推倒重来。

7. Memory 不是“多存一点历史”，而是多层记忆系统

这个项目对记忆的理解很成熟。

它并没有把 Memory 简化成一个存储表，而是把记忆拆成了不同层：

- 短期工作记忆

- 会话级压缩记忆
- 持久记忆
- 记忆整理与 consolidation

7.1 这为什么值得学

因为很多 Agent 项目最容易误判的，就是把“历史消息”当成“记忆”。

但历史消息只是原始材料，不是可复用记忆。

真正可用的记忆系统，至少要回答：

- 什么值得记
- 什么不该记
- 用什么结构记
- 什么时候召回
- 什么时候淘汰或合并

7.2 extractMemories 和 autoDream 提醒了什么

这两个机制说明，记忆至少分两个时间尺度：

- 近端提取：从刚结束的会话里提炼可保留信息
- 远端整理：跨多个会话重写、合并、压缩、升级知识

这个分层非常值得迁移到别的项目。

因为只做提取，不做整理，记忆会越来越碎。

只做整理，不做稳定提取，根本没有足够素材可整理。

7.3 给其他项目的建议

以后如果我给别的 Agent 设计 Memory，我会优先建议他们先定四件事：

1. 记忆 taxonomy，也就是类型体系
2. 写入边界，也就是什么能写、谁能写
3. 召回触发条件
4. 整理与淘汰机制

这四件事比“选哪个数据库”更重要。

7.4 什么时候文件型 memory 很适合

这个项目使用“文件 + 索引”结构，对代码 Agent 很适合。

它适合这些场景：

- 记忆需要人可读、可审查
- 团队想直接看到 Agent 学到了什么
- 知识量还没大到必须上复杂数据库

- 需要和代码仓库、规则文档天然结合

但如果后续进入超大规模、多租户、高频实时检索场景，文件型结构可能就不够了。

8. 多 Agent 最难的不是“多开几个模型”，而是责任闭环

这个项目对多 Agent 的处理很有代表性。

它没有停留在“支持 spawn subagent”这一层，而是继续往下做了：

- Coordinator
- Worker / subagent
- teammate / swarm
- mailbox / notification / task state

8.1 为什么责任闭环比 spawn 更重要

多 Agent 一旦进入真实任务，马上会碰到这些问题：

- 谁负责拆任务
- 谁定义边界
- 谁追踪中间状态
- 谁整合结论
- 谁对用户最终负责

如果这些问题没有答案，多 Agent 只会让系统更乱。

8.2 这个项目给出的好启发

它让我们看到，多 Agent 真正成熟的做法应该包含：

- 角色化，不是每个 Agent 都平权
- 消息化回流，不是只有函数返回值
- 状态可见，不是子 Agent 干完才知道发生了什么
- 生命周期管理，不是起了就不管

8.3 给其他项目的建议

以后我只会在这些情况下建议上多 Agent：

- 任务天然可分工
- 角色边界清晰
- 单 Agent 上下文负担已经过重
- 需要并行探索多个子问题

而在这些情况下，我反而会建议先别上：

- 主链路都还不稳定

- 单轮工具调用都还没治理好
- 没有统一消息协议
- 没有中断、恢复、汇总机制

因为这时多 Agent 往往只是把问题放大。

9. 成本、缓存、延迟，不是上线后再考虑，而是架构问题

这个项目一个非常成熟的地方，是它没有把成本优化理解成后期运维问题。

比如 Prompt cache、forked agent 的 cache-safe 参数、上下文拆分，这些都说明：

成本、延迟、缓存命中，其实应该在架构层就被考虑。

9.1 为什么这件事常被忽略

因为早期 demo 很少暴露这个问题。

但一旦进入真实使用，很快会遇到：

- 每轮上下文越来越贵
- side task 太多导致整体延迟拉长
- 子 Agent 复制太多前缀，浪费缓存机会
- 小改一点动态 Prompt，缓存就全部失效

9.2 给其他项目的建议

以后只要项目出现下面任意一个特征，我就会建议开始做成本工程：

- 长会话
- 高频工具调用
- 后台子任务
- 多 Agent 协作
- 大量重复的系统前缀

这时要考虑的就不只是“模型选便宜一点”，而是：

- 前缀缓存怎么保
- 动态上下文怎么拆
- 哪些任务该 fork
- 哪些任务该延后到后台

10. 工业化 Agent 系统最容易被低估的是“中间层”

如果把一个 Agent 系统粗暴拆成“模型层”和“业务层”，很多真正决定成败的东西都会被忽略。

这个项目最值得借鉴的一点，是它把很多关键工程能力都放在了中间层：

- Prompt 组装层

- Tool 编排层
- Compact 层
- Memory 提取与整理层
- Agent 协调层

10.1 为什么中间层最关键

因为它们负责把“局部能力”变成“系统能力”。

举例来说：

- 有 Prompt，不代表有稳定上下文系统
- 有工具，不代表有可治理执行系统
- 有历史记录，不代表有记忆系统
- 能起子 Agent，不代表有多 Agent 协作系统

这些差距，大多不是模型决定的，而是中间层决定的。

10.2 给其他项目的建议

以后我在评估一个 Agent 项目时，不会只看：

- 模型是不是最新
- 工具数量多不多
- UI 漂不漂亮

我会更看重：

- 中间层有没有清晰边界
- 不同能力是否通过统一协议挂回主循环
- 长会话和副作用有没有治理机制

因为这些东西才决定这个项目能不能从 demo 长成产品。

11. 一份可直接复用的 Agent 设计检查表

以后如果要给别的 Agent 项目提建议，我会优先按这份清单检查。

11.1 Runtime

- 是否存在明确的主执行循环，而不是只有模型调用入口
- 一轮请求的输入、行动、回流、结束条件是否清楚

11.2 Prompt

- 稳定规则和动态上下文是否拆分
- Prompt 是否按片段供给，而不是一段巨型文本
- 是否考虑缓存命中和调试可见性

11.3 Tool Runtime

- 工具是否有统一定义协议
- 是否有权限层
- 是否区分并发安全和非并发安全工具
- 工具结果是否统一消息化回流

11.4 Workflow

- 是否区分单轮、会话、后台、多 Agent 四种时间尺度
- turn-end hooks 是否清楚
- compact 或等价机制是否存在

11.5 RAG / Knowledge

- 知识是否分类
- 是否先做了结构化检索，而不是盲目上向量库
- 外部资料、规则、经验是否有不同召回路径

11.6 Memory

- 是否定义了记忆类型
- 是否有写入边界
- 是否有召回条件
- 是否有 consolidation / 淘汰机制

11.7 Multi-Agent

- 是否有 coordinator 或等价责任中心
- 子 Agent 的结果是否可回流、可观测、可中断
- 是否真的需要多 Agent，而不是单 Agent 没做好

11.8 Cost Engineering

- 是否考虑 Prompt cache
- 是否控制动态上下文波动
- 是否把高成本任务拆到后台或 side task

12. 最后的结论：以后做 Agent 建议时，我最看重什么

如果把这份项目给我的启发压缩成最核心的几句话，就是：

1. 不要把 Agent 做成“会调工具的大模型”，而要做成“可持续工作的运行系统”。
2. 不要把 Prompt 当成一段文本，而要当成上下文供给架构。
3. 不要把 Tool Calling 当成功能演示，而要当成副作用治理系统。

4. 不要把 Workflow 当成步骤图，而要当成跨时间尺度的编排系统。
5. 不要把 Memory 当成聊天记录，而要当成长期知识运营机制。
6. 不要把多 Agent 当成堆模型实例，而要当成责任、状态、结果的组织系统。

以后如果我给别的 Agent 项目提建议，最优先补的，往往不是“再换一个更强模型”，而是：

- 主循环有没有站住
- 中间层有没有补齐
- 状态和副作用有没有治理
- 长期知识有没有沉淀机制

因为这些，才是真正决定 Agent 工程上限的地方。